

Arctic-Embed: Scalable, Efficient, and Accurate Text Embedding Models

Luke Merrick^{1,*}, Danmei Xu¹, Gaurav Nuti¹, and Daniel Campos¹

¹Snowflake Inc.

*Corresponding author, luke.merrick@snowflake.com

Abstract

This report describes the **training dataset creation and recipe** behind the family of **arctic-embed** text embedding models (a set of five models ranging from 22 to 334 million parameters with weights open-sourced under an Apache-2 license). At the time of their release, each model achieved state-of-the-art retrieval accuracy for models of their size on the MTEB Retrieval leaderboard,¹ with the largest model, arctic-embed-l outperforming closed source embedding models such as Cohere’s embed-v3 and Open AI’s text-embed-3-large. In addition to the details of our training recipe, we have provided several informative ablation studies, which we believe are the cause of our model performance.

1 Introduction

Embedding models’ ability to provide accurate retrieval performance without additional tuning (Lewis et al., 2020) has made them a popular choice in search and Retrieval-augmented-generation (RAG) (Ram et al., 2023) workloads. Unlike traditional keyword search, embedding models encode information beyond token overlap. This allows embedding systems to represent queries like *How tall is Tom Cruise?* and *Height of the actor who plays Maverick in Top Gun* closely despite having no common words.

Driven by the utility and the widespread adoption of these models, the broader open-source and research community has put forth a constant stream of ever-stronger text embedding models such as E5 (Wang et al., 2022), GTE (Li et al., 2023a), Jina (Günther et al., 2024). The quick experimentation and improvement underpinning these works is, in turn, thanks in part to large-scale open evaluation benchmarks such as MSMARCO (Campos et al., 2016), BEIR (Thakur et al., 2021), and MTEB

¹<https://huggingface.co/spaces/mteb/leaderboard>

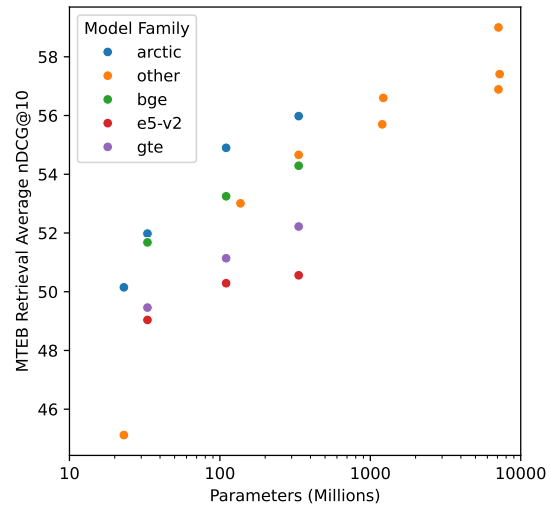


Figure 1: Snowflake’s Arctic-embed models are a suite of 5 embedding models, each of which pushes the Pareto frontier in the trade-off between model size and retrieval performance on the MTEB Retrieval Leaderboard.

(Muennighoff et al., 2023). These leaderboards combine an easy and efficient evaluation with a broad array of tasks, which allows for effective experimentation.

This paper’s work was motivated in early 2024 by the lack of efficient and effective open-text embedding models competing with the performance of closed-source models such as Cohere’s embed-v3 or OpenAI’s text-embed-3-large. While models such as **SFR-Embedding-Mistral** (Yavuz, 2024) and **GritLM** (Muennighoff et al., 2024) outscore proprietary offerings; their size (each over 7 billion parameters) and their dimensionality (each 4096) make them impractical to use in many production workloads. Seeking to provide a high-quality retrieval model with fewer than a billion parameters, we set out to train a suite of high-quality embedding models.

Through fruitful data-centric experiments, we de-

veloped the recently released Arctic family of text embedding models. Based on five encoder-only pretrained language models of various sizes (see Table 1) and leveraging the same training data and methodology, we trained each model to optimize retrieval performance as measured by nDCG@10 on the MTEB Retrieval leaderboard. As shown in Figure 1, each variant achieved a new state-of-the-art performance for its size.² We present these models and this technical report as a journal of our experiments that led to our improvements in performance.

1.1 Summary of Contributions

Open model release. We release a suite of embedding models, Arctic-embed, under a permissive Apache-2 license, which delivers state-of-the-art retrieval performance for their size/context window class on the Retrieval portion of the MTEB leaderboard.

Demonstrated importance of data organization.

We present a set of ablations that suggest improvements in retrieval quality are more strongly tied to data sampling during training and the method of negative mining than scaling up data scale and batch size, where previous work has focused.

Improved methods for synthetic data. We present a novel technique for query generation grounded by mined hard negatives, which we found more effective than straightforward generation approaches that generate both queries and negatives and which served as a key ingredient in our models' success.

2 Background

2.1 Task Description

An embedding model maps a variable input into a fixed-dimensional vector. This one-way transform can be applied to various modalities, scales, and scopes and directly used for downstream tasks such as classification, clustering, or retrieval. In the scope of our work, we focus on text embeddings for retrieval. This task aims to train a model that maximizes the similarity between relevant documents, given a query and a document collection, while minimizing the similarity with irrelevant documents.

The representation-based retrieval method has emerged as a standard paradigm as it minimizes the frequency with which inputs are transformed into vectors. Offline, the document corpus is processed, resulting in a set of vectors stored in an Approximate Nearest Neighbor Index such as FAISS (Douze et al., 2024). The input is transformed online into a vector at query time, and the documents with the closest embeddings are retrieved. In other words, the cosine distance between queries and documents signals relevance.

2.2 Related Work

Training Approaches: Building on prior success in NLP and IR research knowledge, embedding model training uses supervised learning with examples of positive and negative document query pairs. It is common to use labeled positive and negative query-document retrieval examples (commonly extracted from weak signals or labeled data (Lin et al., 2020)) to fine-tune general-purpose pretrained language models into specialized text embedding models. In this paradigm, Qu et al. (2021) demonstrated the importance of scaling the batch size and training on hard negatives. In contrast, Xiong et al. (2020) demonstrated the importance of adapting the negatives' difficulty to the retriever's competence.

While earlier work focused on leveraging supervised datasets such as HotpotQA (Yang et al., 2018) or NQ (Kwiatkowski et al., 2019), Wang et al. (2022) demonstrated the effectiveness of constructing large datasets from web-crawled title-document examples through the groundbreaking performance of their resulting E5 model. Xiao et al. (2023a) and Nussbaum et al. (2024) combine generated datasets with supervised labeled datasets to improve retrieval performance further.

Model Architecture: Building on the success and utility of the transformer (Vaswani et al., 2017) prior work has primarily focused on training models using BERT (Devlin et al., 2018), or variants thereof. While some work has studied the usage of sequence to sequence (Zhuang et al., 2022) or large decoder-only models (Yavuz, 2024) (Muennighoff et al., 2024), these models' increased model size and associated worse inference efficiency have kept the majority of focus on encoder-only variants.

Training Objective: Many works initially trained retrievers and rankers leveraging traditional loss forms such as Mean Squared Error (Lin et al.,

²As of April 16th, 2024

Size	Base Model (Huggingface ID)	Parameters (M)	Embedding Dimension
xs	nreimers/MiniLM-L6-H384-uncased	23	384
s	intfloat/e5-unsupervised-small	33	384
m	intfloat/e5-unsupervised-base	110	764
m-long	nomic-ai/nomic-embed-text-v1-unsupervised	137	768
l	intfloat/e5-unsupervised-large	334	1024

Table 1: Breakdown of model architectures.

2020). Still, recently, the application of a contrastive loss (Hadsell et al., 2006; Mueller and Thyagarajan, 2016), which leverages not only positive pairs but the relationship between positive and negative pairs, has risen to prominence. InfoNCE (Noise Contrastive Estimation) (van den Oord et al., 2018) improved on the contrastive triplet loss and has quickly become one of the most popular and common losses used to train embedding models.

3 Arctic Embed

With Arctic-embed, we aimed to start from the current consensus of best practices from the literature and train an embedding model from the ground up.

Consistent with prior works, like E5, BGE, GTE, Jina, and Nomic (Wang et al., 2024; Xiao et al., 2023b; Li et al., 2023b; Günther et al., 2024; Nussbaum et al., 2024), we conduct two training rounds using two different kinds of datasets. The initial training round is **large-scale pretraining using only in-batch negative examples**. This round of training leverages a dataset of pairs of queries and relevant documents. The second round of training (often referred to as the fine-tuning step) calls for **similar pairs of queries and documents augmented with an additional set of “hard” negative documents** (where “hard” refers to the fact that it is not trivial to determine their lower relevance relative to the labeled-as-relevant document). We used a tunable negative mining strategy (see Section 3.3) to construct a focused dataset of about **a million samples** for this round of training.

Although our work closely replicates many of the steps prior works took, our resulting models score higher on the MTEB Retrieval benchmark, sometimes by a substantial margin. In Table 2 we present several hypotheses about what led to this improved performance, and in Section 7 we test several of these hypotheses through ablation studies.

3.1 Model Architecture

We trained models of varying sizes from BERT-like backbones as shown in Table 1. Our m and l are standard BERT architecture (Devlin et al., 2019) (BERT base and large, respectively). We looked to variants of the MiniLMv2 architecture (Wang et al., 2021) for our smaller sizes (xs and s), and we opted for the Nomic BERT architecture (Nussbaum et al., 2024) for our long-context variant (m-long).

3.2 Pooling

Architecturally, we do not modify any base model, even just the common practice of adding a pooling layer to the base model.³ Additionally, instead of pooling output vectors, we utilize the final hidden state of the **[CLS] token** as the embedding vector, in contrast to the mean pooling strategy used in E5, GTE, and Nomic (Wang et al., 2024; Li et al., 2023b; Nussbaum et al., 2024). This choice matches the BGE architecture (Xiao et al., 2023b) and is inspired by the ablation study in (Li and Li, 2023), showing this led to a 2.5% higher score on the Semantic Text Similarity (STS) evaluation studied.

3.3 Training Data

In creating our training datasets, we took inspiration from the world of Large Language Models (LLMs) and leveraged filtering methods inspired by RefinedWeb (Penedo et al., 2023), C4 (Rae et al., 2022), Gopher (Raffel et al., 2023), and TogetherAI (Computer, 2023).

First, for noisy raw data sources such as web search, we parse structured web documents using **trafilatura**⁴. While parsing, we compute custom signals for quality filtering. Specifically for positive data pair cleaning, we need to ensure: a) each text in the pair is of good quality (language filter, text quality filter) and b) text pairs (query, document)

³e.g., we use `AutoModel.from_pretrained(..., add_pooling_layer=False)` in the transformers Python package)

⁴<https://trafilatura.readthedocs.io/en/latest/>

Difference	Description	Ablation Study
Better data	We leverage <u>web search data</u> and <u>common web data filtering methods</u> .	Shows Improvement
Source stratification	Discussed further in Section 4.4, we fill <u>each minibatch of training data with examples from a single data source</u> . Nomic also uses the technique.	Shows Improvement
Longer pretraining sequence length	We used a maximum sequence <u>length of 256</u> , which is twice as long as in GTE and BGE (Li et al., 2023b; Xiao et al., 2023b), despite using approximately the same batch size as these works.	Shows Improvement
Base models trained for retrieval	Several sizes of Arctic-embed use pre-trained text embedding models as starting points. For example, we use e5-unsupervised-base instead of its parent model bert-base-uncased as the backbone of arctic-embed-m.	Shows Inconsistent Improvement
[CLS] embeddings	Unlike most prior art, Arctic embed uses [CLS] token embeddings instead of mean pooling. BGE also uses the technique.	Not Studied
Implementation and tuning	We iterated relentlessly on the data mix, negative mining strategy, batch size, and other training parameters to double down on our data advantages in both training rounds. Additionally, our training implementation includes several attention-to-detail tricks (e.g., in-batch document deduplication), which may improve performance compared to more naive implementations.	Not Studied

Table 2: Differences from prior works hypothesized to help Arctic embed score higher on MTEB Retrieval.

are similar in meaning (consistency filter). For quality filtering, we leverage a series of filters similar to ones detailed in Snowflake’s Arctic model training cookbook⁵. A complete list of effective filtering methods can be found in Appendix C. We combine these filters to create a more curated dataset by removing low-quality, irrelevant, or potentially spam documents based on various characteristics related to content quality, language structure, and duplication.

For consistency filtering, we apply a low-fidelity, high-throughput pair-similarity consistency filter — sentence similarity using a fastText⁶ word2vec model (which can be run cheaply on CPU). Rather than treating these embeddings’ signal as a clear quality label, we instead adopt a conservative threshold (a low minimum allowed similarity of 0.3) and use them to filter out unrelated examples (e.g., “CGplayer doesn’t work properly without JavaScript-enabled” documents from web crawl failures). Additionally, we truncate long sequences to 512 words during this step. As we observed, queries in the web-based corpus were usually answered at the beginning of the document. Not only was it computationally wasteful, but even the meaning captured in word2vec embeddings would get diluted by averaging vectors from irrelevant words present later.

⁵<https://medium.com/snowflake/snowflake-arctic-cookbook-series-arctics-approach-to-data-b81a8a0958bd>

⁶<https://fasttext.cc/docs/en/english-vectors.html>

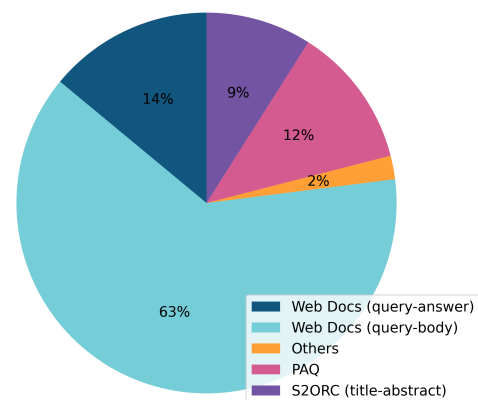


Figure 2: Composition of our pretraining dataset containing about 300 million query document pairs.

3.4 Dataset Mix And Sampling

Due to the different datasets’ sizes, consistency, hardness, and learning dynamics, simply concatenating all available datasets together proved a sub-optimal strategy, especially in the fine-tuning stage. Instead, we ran isolated experiments to understand the effects of each dataset on fine-tuned performance. Then, we selected and combined datasets based on their relative performance in these experiments. Each data source we used is described in more depth below.

Our large pretraining dataset, described in figure 2, amounts to 308 million query-document pairs (filtered from around 2 billion documents), of which 71% are web search documents paired with either a query or title. Aside from web search

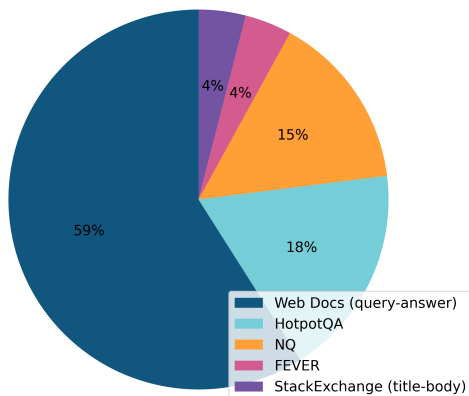


Figure 3: Composition of our finetuning dataset, which contains about 1 million queries, each paired with a positive document and a set of hard negative documents.

data, text pairs set include PAQ⁷, StackExchange title-body and title-body web documents pairs from common crawl based sources, and S2ORC title-abstract pairs⁸. We have found the previous steps on quality annotation and filter **transformative** in improving quality and pruning noise in web search data and beyond for pairwise positive datasets.

Our fine-tuning dataset, described in Figure 3, consists of around **1 million pairs** built by combining our web search data with several public datasets (HotpotQA⁹, NQ¹⁰, Fever¹¹, and StackExchange title-body¹²), and then performing further expansion via synthetic mining strategy detailed in section below. This mix notably omits several popular public datasets used by other embedding models due to our observation of **positive pair consistency and negative pair level of hardness**. These **found-to-be-less-useful** datasets include NLI, MEDI, WikiAnswers, and SQuAD. Empirically, we have observed that **quantity is less important than quality in the finetuning phase**, and an overpowering amount of low-quality data can lead to lower-quality models.

3.5 Synthetic Data For Semantic Dense Mining

Compared to the abundance of web-scale data used in pretraining, high-quality examples suitable for

⁷<https://github.com/facebookresearch/PAQ>

⁸<https://github.com/allenai/s2orc>

⁹<https://github.com/hotpotqa/hotpot>

¹⁰<https://github.com/google-research-datasets/natural-questions>

¹¹<https://fever.ai/dataset/fever.html>

¹²<https://huggingface.co/datasets/sentence-transformers/embedding-training-data>

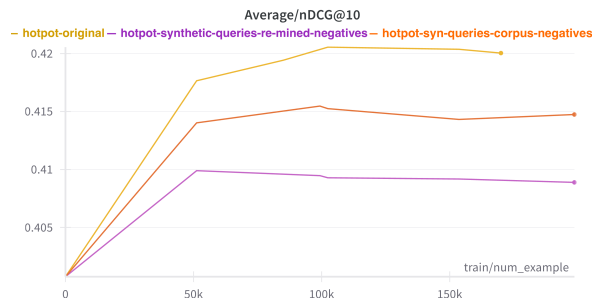


Figure 4: Comparison of model performance when training using synthetic datasets generated from the HotpotQA document corpus versus the original HotpotQA queries (using Algorithm 1 for negative mining in both cases).

finetuning are more scarce. To address this data scarcity, we used **synthetic data creation to construct additional datasets that benefited downstream performance just as much as those listed above**. Similar to the prior work of Dai et al. (2022); Lee et al. (2024), we leverage Large Language Models to generate novel queries. Breaking from these previous approaches, however, we found **it critical to add negative documents to our LLM inputs to ground the query generation** (see Algorithm 2 in the Appendix for details). Additionally, we chose to **generate only synthetic queries rather than synthetic negatives** because we found that LLMs do not easily generate relevant negatives of as high quality as those mined from a preexisting corpus of documents. Figure 4 shows this approach in action – two datasets generated by variants of Algorithm 2 led to score increases approaching that afforded by the original HotpotQA.

3.6 Tunable Hard Negative Mining

Fine-tuning datasets typically include carefully chosen “hard” negative examples and a positively-relevant query-document pair. How hard should these negatives be for maximally effective learning in the fine-tuning phase? Our answer to this question was ultimately **a tunable hard negative mining strategy in which we leveraged a preexisting text embedding model to identify and score the hardest negatives for each training example**. Then, we applied a score threshold to discard the hard negatives from the above set. We found that **using an upper threshold rather than a specific rank helped** account for the fact that some queries admit much harder top-k negatives than others, and in Section 7.2, we perform a parameter sweep of the negative hard-

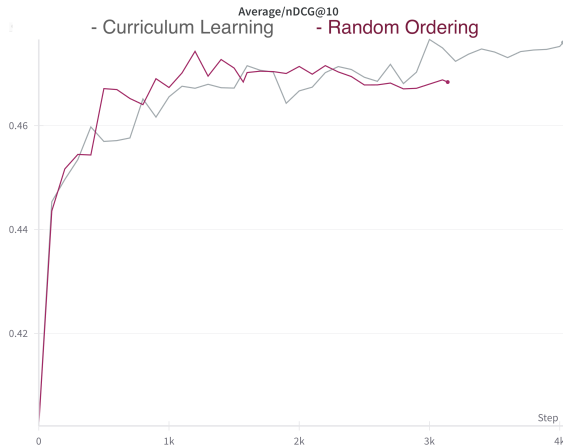


Figure 5: Impact of curriculum learning on nDCG@10 retrieval score during training. Scores averaged over a handful of small datasets used for fast in-training evaluation.

Variant	Pre Batch	Pre LR	Finetune Batch	Fine LR
xs	24,576	6e-4	768	4e-5
s	32,768	5e-4	1,024	4e-5
m	16,384	2e-4	512	1e-5
m-long	12,288	1e-4	512	1e-5
l	12,480	1e-4	512	9e-6

Table 3: Learning rate and batch size for both rounds of training.

ness threshold to demonstrate the value of a tunable approach (the optimal threshold value scores significantly better than other choices). We additionally note that although Algorithm 1 indicates both an upper and lower relevance threshold for negative mining, in practice, we retrieved the top 100 hardest negatives and applied only an upper threshold as a performance optimization.

Beyond tuning to a single hardness threshold level, we hypothesized that ordering the data by the difficulty of the negatives (i.e., curriculum learning) could lead to even better results. In this vein, we offer the experiment shown in Figure 5, which compares the Impact of training with negatives of progressively increasing difficulty. While this initial experiment suggests some improvement in curating the curriculum of hard negatives, we note that this experiment was run after the release of the Arctic embed, and we did not use this curriculum approach when training our published models.

4 Training Recipe

4.1 Model Initialization

We begin with a pretrained language model. Where permissively licensed base models pre-trained for

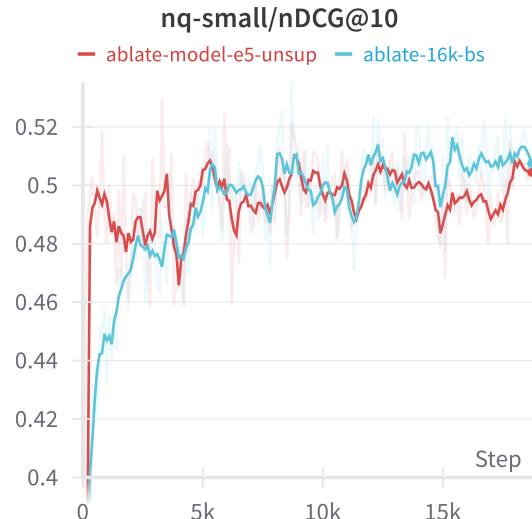


Figure 6: A granular look at our starting weight ablation study from Section 7.1. We plotted the rolling average nDCG@10 score throughout training, observing that the run using pre-trained E5 model weights (red) converged much more quickly than the run using general-purpose BERT weights (blue). The evaluation dataset is a “lite BEIR” dataset based on NQ – details in Appendix B.1.

information retrieval are available for a given model size, we prefer these weights over general-purpose pretrained ones¹³. Our ablation studies in Sections 7.1 and 7.3 showed mixed results and suggested the effect of this design choice on performance may have been weak relative to other effects studied, but as Figure 6 shows, starting from a more thoroughly trained base model, such as e5-base-unsupervised, had an apparent effect on sample-efficiency and convergence speed, and this speedup was notably helpful for faster experimentation during model development.

4.2 Large Scale Contrastive Pretraining With In-Batch Negatives

In the first round of contrastive training, we aim for large scale, both in batch and total dataset sizes. We use our pretraining dataset with the infoNCE contrastive loss using in-batch negatives (for each query, all documents associated with different queries in the minibatch are treated as negative examples). GPU parallelism, activation checkpointing, and truncated sequence length were instrumental in achieving large batch sizes.

¹³Such as <https://huggingface.co/intfloat/e5-large-unsupervised>

We train for one epoch¹⁴ using the AdamW optimizer, adjusting only the learning rate while leaving all other parameters at PyTorch default values. We perform a linear learning rate warmup for several hundred steps, then a linear decay to 10% of the original learning rate over the remainder of the training. As evidenced by the example shown in Figure 10, we observed performance could be sensitive to the learning rate and the learning rate schedule. Batch sizes and learning rates for each model size are given in Table 3.

4.3 Longer Truncation Length

Much of our pretraining data included documents significantly longer than 128 tokens. We used a document sequence length of 256 in large-scale contrastive training, in contrast to the 128 truncation length used in GTE and BGE. We truncated query sequence length to 32, consistent with BGE’s source code¹⁵. Our ablation study in Section 7 suggests this longer truncation length led to a substantial improvement in retrieval performance.

4.4 Source Stratification

We fill each batch with data from a single source during pretraining, a source of accuracy gains in prior work (Nussbaum et al., 2024). Our ablation study in Section 7.1 indicates this led to a dramatic improvement in model quality (see Table 5).

4.5 Quality-Focused Contrastive Training With Curated Negatives

After large-scale training, we perform a second round of training leveraging our fine-tuning dataset, which contains explicitly labeled negative examples. We use no learning rate warmup but apply the same linear learning rate decay schedule as in the pretraining stage. We truncate sequence lengths to 512 for queries and documents for all models, including the long-context variant m-long. For each query in a batch, we include one positive document and ten hard negative documents. Batch sizes (number of queries) and learning rates for each model size are given in Table 3.

¹⁴Some sizes of Arctic embed utilized early checkpoints from before one epoch of pretraining, though this was done for expediency, and we did not find evidence of this improved performance.

¹⁵https://github.com/FlagOpen/FlagEmbedding/blob/53cfac4a50ac0e023b2f8d19b10667c9c210fa41/FlagEmbedding/baai_general_embedding/finetune/arguments.py

4.6 Disabling In-Batch Negative Loss

Based on some early fine-tuning runs, we found that disabling in-batch negative loss did not measurably degrade performance. We stopped using in-batch negatives during fine-tuning (this made tuning easier, especially since the interaction between batch size and in-batch loss is not straightforward).

5 Efficiency

To maximize experimental throughput, iteration speed, and the maximum feasible batch size, we took pains to ensure our training setup was as effective as possible for our given computational budget. We carefully optimized the net efficiency of training and iteration, assuming a single training node with 8 NVIDIA H100 GPUs. We achieved high efficiency by carefully implementing a custom data loader and writing our training loop in plain PyTorch to leverage several “tricks” we detail in Appendix B.2. Additionally, we identified and eliminated careless performance bottlenecks through performance benchmarking, keeping a watchful eye on both throughput and GPU utilization.

Further discussions about methods we found helpful for efficient experimentation and training can be found in the Appendix, including a discussion of granular evaluation during training in Appendix B.1.

6 Experimental Results

To qualify our retrieval quality, we evaluate model performance on the Retrieval portion of the MTEB dataset (Muennighoff et al., 2023). Summary results of MTEB experiments are shown in Figure 1, and a complete tabulation by dataset is given in Appendix E. To qualify the performance of our long context model, we leverage the LoCo (Saad-Falcon et al., 2024), the results of which are given immediately below.

6.1 Long Context Performance

For our initial Arctic embed release, we did not put any special efforts into adjusting our training recipe for long-context support. Instead, our m-long variant was only trained on short sequence data (it was pretrained with sequences truncated to 256 tokens and finetuned with sequences truncated to 512 tokens). Nonetheless, even on the specialized LoCo long context benchmark datasets (Table 4), performance only tends to lag slightly compared

Model	Seq Len	Summ. Scr. FD	Gov. Report	QMSUM	QASPER Title	QASPER Abs.	Avg
arctic-embed-m-long	2048	63.2	93.8	45.8	77.3	95.8	75.2
arctic-embed-m-long	4096	81.8	96.0	39.7	85.9	99.0	80.5
arctic-embed-m-long	8192	86.6	96.5	31.7	81.6	99.7	79.2
jina-base-v2	2048	87.2	97.7	35.1	95.3	99.7	83.0
jina-base-v2	8192	93.3	98.6	40.8	95.1	99.3	85.5
nomic-embed-text-v1	2048	86.1	96.9	47.8	96.1	99.7	85.3
nomic-embed-text-v1	4096	89.0	97.4	45.7	95.8	99.9	85.6
nomic-embed-text-v1	8192	90.9	97.8	44.2	94.9	99.9	85.5
e5-mistral	4096	95.9	98.3	46.8	98.4	99.8	87.8

Table 4: Snowflake-arctic-embed-m-long nDCG@10 scores on the LoCo benchmark. Non-Arctic scores taken from (Nussbaum et al., 2024) without an attempt to reproduce.

to models trained end-to-end specifically with long-context in mind, e.g. nomic-embed-text-v1. While these LoCo results suggest m-long may not be the model of choice for long sequences, its strong MTEB Retrieval scores suggest it may be a good pick for datasets containing a mix of long and short sequences.

This surprisingly not-so-bad performance may be largely thanks to the base model of m-long, nomic-embed-unsupervised, being trained on long sequence retrieval, but unfortunately we did not have time to run an ablation study to quantify the Impact of this base model.

7 Ablation Studies

We conducted several ablation studies to test some of the hypotheses stated in Table 2 regarding the causes of Arctic embed’s higher MTEB Retrieval scores relative to other similar models. Average scores are given in the following subsections, with full scores in Appendix E.

7.1 Pre-training Ablations

We probed the effects of batch size, sequence length, base model, and training data in a series of ablations, with resulting MTEB Retrieval scores tabulated in Table 5. In each case we trained for 20k steps using a linear learning rate decay from $2e-4$ to $2e-5$ after a 300-step linear warmup from 0.¹⁶

Overall, the ablation study results support our hypotheses about data sourcing, longer sequence

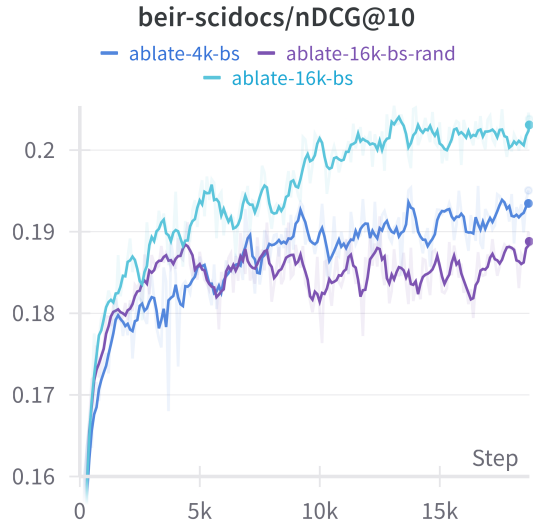


Figure 7: A granular look at our source stratification ablation study (also see Table 5) showing rolling average nDCG@10 on the SciDocs dataset. A large batch size and source stratification (light blue) delivers the highest performance. Although the random-source large batch size run (purple) drives performance up sharply at the start of training, without source stratification, performance quickly plateaus, falling behind the source-stratified small batch run (dark blue) despite using $4\times$ the data and compute.

¹⁶This ablation setup is slightly different from our published models’ configuration – the warmup was 300 steps instead of 100, gradient clipping was enabled, and 20k steps often slightly exceeded the one epoch through the data used in our published models (often one epoch was around 19k steps). In some cases, we evaluated a one-epoch checkpoint (around 19k steps instead of 20k) to mitigate a data loading correctness issue discovered post-training for the beyond-one-epoch regime for this dataset.

Run	Base Model	Data	Stratify Source	Batch Size	Seq. Length	Score
A	bert-base-uncased	Snowflake	Yes	16,384	256	46.97
B	e5-unsupervised-base	Snowflake	Yes	16,384	256	46.96
C	bert-base-uncased	Nomic	Yes	16,384	256	46.55
D	bert-base-uncased	Snowflake	No	16,384	256	43.74
E	bert-base-uncased	Snowflake	Yes	4,096	256	45.36
F	bert-base-uncased	Snowflake	Yes	16,384	128	45.53

Table 5: Large-scale training ablation study. Varied treatment bolded. The score is nDCG@10 on the MTEB Retrieval benchmark.

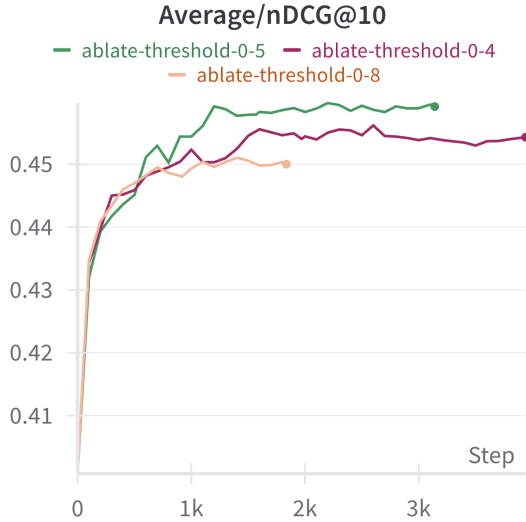


Figure 8: Ablation study for different threshold for hard negatives

length, and source stratification improving model performance. In contrast, the choice of initializing from a pre-trained retrieval model did not significantly impact the MTEB Retrieval score after pre-training. We also notice the interesting curriculum-learning-like pattern of source stratification maturing more later in training than other factors like batch size (see Figure 7).

7.2 Fine-tuning Ablations

As discussed in Section 3.6, our tunable negative mining approach uses a threshold to filter out too-hard negatives. We perform an ablation study on several threshold values to demonstrate the importance of the threshold parameter. The results shown in Figure 8 indicate that too-low and too-high maximum relevancy thresholds (too-hard and too-easy negatives) lead to significantly worse performance.

7.3 End-to-end ablations

To thoroughly study the effect of training data on the final score, we extended a subset of our

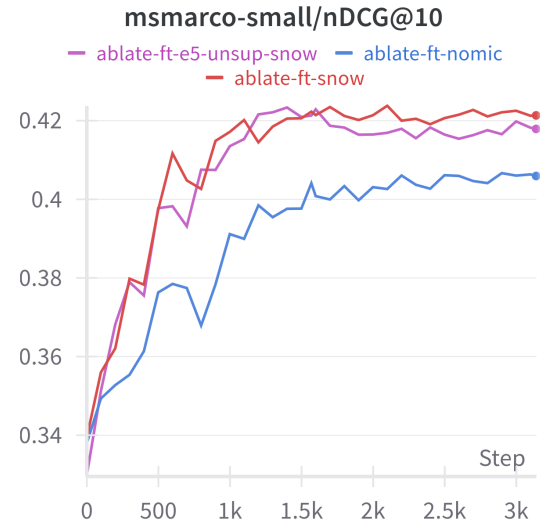


Figure 9: Comparison of various pretrained models under during identical fine-tuning. The performance gap associated with different pretraining datasets appears rather quickly, while the gap associated with different base model weights does not even appear for this dataset (dataset details in Appendix B.1).

Starting Model	Pretrain Data	Score After Pretrain	Final Score
bert-base	Snow	46.97	53.92
e5-unsup.	Snow	46.96	54.67
bert-base	Nomic	46.55	52.23

Table 6: Final two-stage training scores for our published m model two ablations.

pretraining ablation study through the fine-tuning step. We conducted a finetuning step similar to the one used on our published arctic-embed-m model on configurations A, B, and C from Table 5 (different data and base model). The pretraining and fine-tuning trajectories are shown in Figure 9, with final MTEB Retrieval scores in Table 6. Although the performance gap between models pre-trained with Snowflake and Nomic data was relatively modest in pretraining, the gap widens substantially with fine-tuning, despite the fine-tuning recipe is the same. We also see a slight improvement in the final score for the configuration using e5-unsupervised-base. We note that our tuning the fine-tuning step to an e5-unsupervised-base model pre-trained on our data may have affected these results.

8 Conclusion and Future Work

By creating the suite of Arctic text embedding models, we sought to better understand how to optimize the training recipe for high-quality text embedding models. Our exploration found that dataset-stratified mini-batches and tuned hard negative mining were crucial ingredients for training a model for more effective retrieval.

In the future, we seek to continue our experimentation to leverage improved curriculum learning and better methods of source stratification. Additionally, we strive to train more robust models to compression approaches such as binarization or quantization of embeddings.

References

- Daniel Fernando Campos, Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, Li Deng, and Bhaskar Mitra. 2016. Ms marco: A human generated machine reading comprehension dataset. *ArXiv*, abs/1611.09268.
- Yihao Chen, Xianbiao Qi, Jianan Wang, and Lei Zhang. 2023. [Disco-clip: A distributed contrastive loss for memory efficient clip training](#).
- Together Computer. 2023. [Redpajama: an open dataset for training large language models](#).
- Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. 2022. [Promptagator: Few-shot dense retrieval from 8 examples](#).
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#).
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. [The faiss library](#).
- Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2024. [Jina embeddings 2: 8192-token general-purpose text embeddings for long documents](#).
- Raia Hadsell, Sumit Chopra, and Yann LeCun. 2006. [Dimensionality reduction by learning an invariant mapping](#). 2006 *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, 2:1735–1742.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc V. Le, and Slav Petrov. 2019. [Natural questions: A benchmark for question answering research](#). *Transactions of the Association for Computational Linguistics*, 7:453–466.
- Jinhyuk Lee, Zhuyun Dai, Xiaoqi Ren, Blair Chen, Daniel Cer, Jeremy R. Cole, Kai Hui, Michael Boratko, Rajvi Kapadia, Wen Ding, Yi Luan, Sai Meher Karthik Duddu, Gustavo Hernandez Abrego, Weiqiang Shi, Nithi Gupta, Aditya Kusupati, Praateek Jain, Siddhartha Reddy Jonnalagadda, Ming-Wei Chang, and Iftexhar Naim. 2024. [Gecko: Versatile text embeddings distilled from large language models](#).
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates Inc.
- Xianming Li and Jing Li. 2023. [Angle-optimized text embeddings](#).
- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023a. [Towards general text embeddings with multi-stage contrastive learning](#). *ArXiv*, abs/2308.03281.

- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023b. [Towards general text embeddings with multi-stage contrastive learning](#).
- Jimmy J. Lin, Rodrigo Nogueira, and Andrew Yates. 2020. [Pretrained transformers for text ranking: Bert and beyond](#). *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*.
- Jonas W. Mueller and Aditya Thyagarajan. 2016. [Siamese recurrent architectures for learning sentence similarity](#). In *AAAI Conference on Artificial Intelligence*.
- Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. [Generative representational instruction tuning](#).
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. [Mteb: Massive text embedding benchmark](#).
- Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. 2024. [Nomic embed: Training a reproducible long context text embedder](#).
- Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. [The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only](#).
- Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. [Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering](#).
- Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Buden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimppoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d’Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorraine Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. 2022. [Scaling language models: Methods, analysis insights from training gopher](#).
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. [Exploring the limits of transfer learning with a unified text-to-text transformer](#).
- Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. [In-context retrieval-augmented language models](#). *Transactions of the Association for Computational Linguistics*, 11:1316–1331.
- Jon Saad-Falcon, Daniel Y. Fu, Simran Arora, Neel Guha, and Christopher Ré. 2024. [Benchmarking and building long-context retrieval models with loco and m2-bert](#).
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. [BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models](#). In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- Aäron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. [Representation learning with contrastive predictive coding](#). *ArXiv*, abs/1807.03748.
- Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). *ArXiv*, abs/1706.03762.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. [Text embeddings by weakly-supervised contrastive pre-training](#). *ArXiv*, abs/2212.03533.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2024. [Text embeddings by weakly-supervised contrastive pre-training](#).
- Wenhui Wang, Hangbo Bao, Shaohan Huang, Li Dong, and Furu Wei. 2021. [Minilmv2: Multi-head self-attention relation distillation for compressing pre-trained transformers](#).
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023a. [C-pack: Packaged resources to advance general chinese embedding](#).
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023b. [C-pack: Packaged resources to advance general chinese embedding](#).
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and

Arnold Overwijk. 2020. [Approximate nearest neighbor negative contrastive learning for dense text retrieval](#). *ArXiv*, abs/2007.00808.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [Hotpotqa: A dataset for diverse, explainable multi-hop question answering](#). In *Conference on Empirical Methods in Natural Language Processing*.

Rui Meng Ye Liu Shafiq Rayhan Joty Caiming Xiong Yingbo Zhou Semih Yavuz. 2024. [Sfr-embedding-mistral:enhance text retrieval with transfer learning](#). Salesforce AI Research Blog.

Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. 2022. [Rankt5: Fine-tuning t5 for text ranking with ranking losses](#).

A Dataset Creation Algorithm Details

Algorithms 1 and 2 provide the details of the algorithms used to create our high-quality fine-tuning dataset (see Section 3.3).

B Additional Efficiency Details

Table 7 quantifies our training efficiency in terms of throughput.

B.1 Fast Feedback With “Lite” Datasets

We found it valuable to conduct information retrieval evaluation at regular intervals throughout the training process, not just at the end, as this uncovered significant trends not captured by end-of-training evaluation (see Figure 7, for example). To enable these evaluations, we constructed “lite” versions of several large BEIR datasets by starting with a sample of queries (e.g., a few hundred) and combining the labeled-as-relevant with the most relevant documents as determined using a preexisting text embedding model (we often used 100 documents per query) to form a corpus far smaller than the original yet still difficult given the query test set. We found that these “lite” datasets offered a cheap way to anticipate full-scale performance trends at a small fraction of the compute cost required by the entire dataset.

We could evaluate retrieval performance on a diverse set of domains throughout training through a combination of already-small BEIR datasets and our new “lite BEIR” datasets. We achieved excellent throughput by implementing evaluation in the same Distributed Data-Parallel paradigm as our training. We were able to embed and score ~five

datasets of this size in ~30 seconds, netting us useful nDCG@10 scores as often as every 100 steps of training with only modest runtime overhead (Figures 6, 7 and 10, which are screenshots of our in-training evaluation, demonstrate just how granular and proper this evaluation frequency is in practice).

B.2 Efficiency Tricks

Training Efficiency Tricks

- Automatic Mixed Precision¹⁷ offers bfloat16 training speeds (often a ~2x speedup) with little to no quality degradation compared to float32 training.
- PyTorch Distributed Data-Parallel (DDP) includes a ZeroRedundancyOptimizer implementation¹⁸ which can reduce memory consumption, though the savings are modest for typical BERT-scale models.
- Enabling `PyTorch’s` `roundup_power2_divisions` and/or `expandable_segments` CUDA memory allocator parameters¹⁹ can mitigate memory fragmentation issues arising from inconsistent batch size and sequence length (e.g. from batch deduplication or padding each batch to the length of the longest sequence it contains).
- The DisCo trick for distributed contrastive loss (Chen et al., 2023) is just a few extra lines of PyTorch code²⁰.

C List of Data Quality Filter Heuristics

- **Language:** Only include documents primarily identified as English using the fasttext language classifier threshold.
- **Document Length:** The number of normalized words in each document should be between 10 and 10,000. This filter removes very short or extremely long documents.
- **Word Length:** After normalization, the mean length of words should be between 3 and 10 characters. This filter helps remove documents with excessively short or long words.

¹⁷<https://pytorch.org/docs/stable/amp.html>

¹⁸https://pytorch.org/tutorials/recipes/zero_redundancy_optimizer.html

¹⁹<https://pytorch.org/docs/stable/notes/cuda.html>

²⁰<https://github.com/IDEA-Research/DisCo-CLIP/blob/main/disco/gather.py>

Require:

- P , a dataset of m query-document *pairs*
 - $P = \{(q_1, d_1), (q_2, d_2), \dots, (q_m, d_m)\}$
- D , a corpus of n documents
 - $D = \{d_1, d_2, \dots, d_n\}$
 - One option is for D is to reuse the documents from P
- r , a semantic relevance scoring function
 - Maps two pieces of text t_1 and t_2 to a real number, i.e. $\text{score} = r(t_1, t_2)$
 - Should effectively score the semantic relevance of query-query, document-document, and query-document pairs
 - Example implementation: use a preexisting text embedding model to embed the query and document, then compute a vector similarity score via cosine similarity.

Parameters:

- $R_{\max}$, a real-valued maximum relevance threshold value
 - This threshold defines the degree of relevance below which two items can be considered “irrelevant” for the sake of training. This cutoff helps us attenuate label noise and avoid trying to teach the text embedding model to treat relevant documents as irrelevant.
 - $R_{\min}$, a real-valued minimum relevance threshold value
 - This threshold defines the degree of relevance below which two items are considered “too obviously irrelevant” for the sake of training. This cutoff helps us keep all negative examples difficult enough for the model.
 - k_{neg} , the maximum desired number of negatives to mine per query-document pair
-

1. Initialize dataset $Z = \{\}$
 2. For each document $d_i \in D$ do
 - (a) Compute relevance scores $\mathbf{s} = [s_1, \dots, s_n] = [r(q_i, d_1), r(q_i, d_2), \dots, r(q_i, d_n)]$
 - (b) Drop from consideration all documents with relevance scores beyond the minimum or maximum relevance threshold parameters, i.e., define $\mathbf{s}' = [s_i : R_{\min} \leq s_i \leq R_{\max}]$
 - (c) Use the scores to determine the most relevant documents $D_{\text{topk}} = \{d_{i,1}, \dots, d_{i,k}\} = \text{topk}_{\mathbf{s}'}(D)$
 - (d) Update Z with the query-document-documents example $(q_i, d_i, \{d_{i,1}, \dots, d_{i,k}\})$, i.e. $Z_{\text{updated}} = Z \cup \{(q_i, d_i, \{d_{i,1}, \dots, d_{i,k}\})\}$
-

Variations:

- In addition to positive-query-to-negative-document relevance $r(q_i, d_j)$, positive-document-to-negative-document relevance $r(d_i, d_j)$ can also be used as a signal for mining hard negatives.
-

Algorithm 2: Synthetic Data Generation

Require:

- D , a corpus of n documents (same as Algorithm 1)
 - g , a synthetic query generation function
 - Maps one positive (relevant) document and k examples of negative (irrelevant) documents to a synthetic query, i.e. $q = g(d_{\text{pos}}, d_{\text{neg}}^{(1)}, d_{\text{neg}}^{(2)}, \dots, d_{\text{neg}}^{(k)})$.
 - Example implementation: prompt an instruction-tuned LLM using the positive and negative example documents.
 - r , a semantic relevance scoring function (same as Algorithm 1)
-

1. Initialize empty paired dataset $P = \{\}$
 2. For each document $d_i \in D$ do:
 - (a) Identify the top k documents with highest semantic relevance
 - i. Compute relevance scores $\mathbf{s} = [r(q_i, d_1), r(q_i, d_2), \dots, r(q_i, d_n)]$
 - ii. Use the scores to determine the most relevant documents $D_{\text{topk}} = \{d_{i,1}, \dots, d_{i,k}\} = \text{topk}_{\mathbf{s}}(D)$
 - (b) Use generate a synthetic query $q_i = g(d_i, d_{i,1}, \dots, d_{i,k})$
 - (c) Update P with the query pair (q_i, d_i) , i.e. $P_{\text{updated}} = P \cup \{(q_i, d_i)\}$
 3. Apply Algorithm 1 to mine negative documents from paired data P and corpus D using relevance function r
-

Variations:

- In addition to a positive document and a sequence of negative documents, the query generation function can also be constructed to accept an example query to ground the generation. In this case, paired data consisting of query-document pairs becomes required, but higher-quality outputs may be possible in the case of high-quality grounding queries. In this setting, query-document relevance can also be used as a signal for selecting negative examples.
 - Randomization—rather than taking the top k most relevant documents, one can select the top $k' > k$ documents and randomly sample k of these. This approach (and similar randomization approaches) can generate multiple queries from a single document (or query-document pair if the above variation is used).
 - Relevance thresholding – similar to the machinery of Algorithm 1, one can adjust the relevant negative examples used in query generation by discarding examples deemed “too relevant” or “too irrelevant” as per thresholds on relevance score.
-

Algorithm 3: Synthetic Data Generation Prompt

You are a search quality rater tasked with evaluating the effectiveness of a search engine. You aim to generate a plausible query that retrieves a specific document when executed on a high-performing search engine. The query should be relevant to the document’s content and make sense without access to the relevant document. Sample Document: – BEGIN SAMPLE DOCUMENT – SAMPLE_DOC – END SAMPLE DOCUMENT – Sample Generated Query: – BEGIN SAMPLE GENERATED QUERY – SAMPLE_QUERY – END SAMPLE GENERATED QUERY – Document Details: – BEGIN DOCUMENT TEXT – DOCUMENT_TEXT – END DOCUMENT TEXT – Irrelevant Documents: Irrelevant Document 1: WRONG_1 Irrelevant Document 2: WRONG_2 Irrelevant Document 3: WRONG_3 Irrelevant Document 4: WRONG_4.

Instructions: Read the document text and consider potential user actions. Reflect on the document’s content and utility. Review the irrelevant documents to understand query nuances. Create a query (Q) that retrieves the target document but excludes irrelevant ones.

Considerations: Does the query fully address the user’s intent? Does the query uniquely identify the target document?

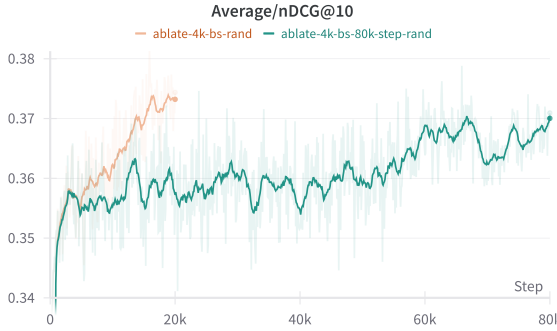
Comprehensive Explanation: Provide a detailed explanation of why the generated query effectively retrieves the target document and excludes irrelevant ones. Ensure the explanation precedes the query for clarity and evaluation purposes.

Respond only with the JSON object comprising keys E and Q.

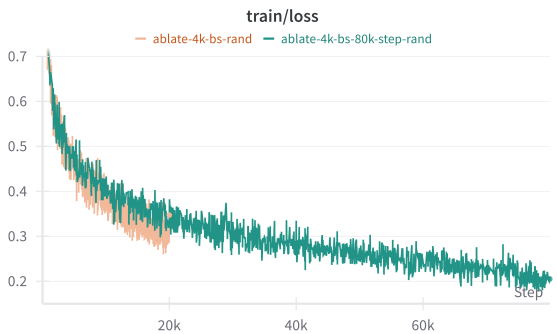
Training State	Batches	Doc/Batch	Doc Max Length	Elapsed	Doc/Sec
Large Scale In-Batch Negatives	18,798	16,384	256	17h3m	5,018
Smaller Scale Hard Negatives	7,845	5,632	512	7h40m	1,601

Table 7: Training efficiency for arctic-embed-m on 8 NVIDIA H100 GPUs. The in-training evaluation is included in the runtime.

- **Symbol Density:** The document’s ratio of symbols to words should be less than 0.1 (10%). This filter removes documents with an excessive number of symbols or special characters.
- **Ellipsis Line:** The fraction of lines that end with an ellipsis (...) should be less than 0.3 (30%). This filter removes documents with an excessive number of lines ending with ellipses.
- **Non-Alphabetical Word:** The fraction of words that contain no alphabetical characters should be less than 0.2 (20%). This filter removes documents with excessive non-alphabetical words (e.g., numeric strings, symbols).
- **Perplexity:** The perplexity score (from <https://github.com/kpu/kenlm>) should be less than 10,000. This filter removes documents that are less likely to be understandable or meaningful.
- **N-Gram Duplication:** Limit the fraction of characters in duplicate n-grams (sequences of n words) for n ranging from 2 to 10. These filters help remove documents with excessive repetition or duplication of phrases.
- **Stop Word:** The document should contain at least one stop word (common words like “the”, “and”, “is”). This filter helps remove documents without meaningful content.
- **Bullet Point Line:** The density of lines starting with bullet points should be less than 0.9 (90%). This filter removes documents that are mostly composed of bullet points or lists.
- **Blacklist Domain:** Domain of document URL cannot be blacklisted based on the https://dsi.ut-capitole.fr/blacklists/index_en.php. This filter helps remove documents from known low-quality, adult, or spam domains.
- **Short Line:** The fraction of lines with fewer than five words should be less than 0.1 (10%). This filter removes documents with an excessive number of concise lines.
- **Numeric Line:** The fraction of lines containing only numeric characters should be less than 0.05 (5%). This filter removes documents with excessive lines consisting solely of numbers.
- **Uppercase Line:** The fraction of lines with more than 80% uppercase letters should be less than 0.05 (5%). This filter removes documents with excessive lines in all uppercase letters.



(a) nDCG@10 score averaged across several datasets



(b) Loss curves

Figure 10: Example of surprising sensitivity to learning rate schedule. The yellow lines track a 20k step training run, while the green lines show the same data and hyperparameters extended to an 80k step run, with the only difference in the first steps being the **slower linear learning rate decay in the longer run**. Although learning rates are still similar at 6k steps (approximately 0.00015 and 0.00019 for yellow and green, respectively), the learning trajectories diverge sharply around this point, with the 20k schedule learning faster both in terms of downstream IR performance (Figure 10a) and in-sample contrastive loss (Figure 10b).

D Anecdota On The Importance Of Tuning

Throughout the development of Arctic Embed, we found the time-honored technique of guess-and-check tuning to be critical for ensuring a quality final model. By investing in efficient training and high-granularity in-training evaluation (see Section 5), we were able to carry out dozens of experiments ranging from one-off wild guesses (“YOLO runs”) to large-scale parameter sweeps spanning ten or more trials in the background of our work to build our in-house datasets and finish our training code implementation.

We were surprised quite often in our tuning. Sometimes, we were surprised by an unex-

Stratify Source	Steps	Score
Yes	20k	45.36
Yes	80k	46.13
No	20k	42.42
No	80k	42.79

Table 8: Extended study of 4k batch size runs demonstrating better performance at fewer steps. A one-epoch (75k step) checkpoint is used in the 80k stratified due to a data loading bug.

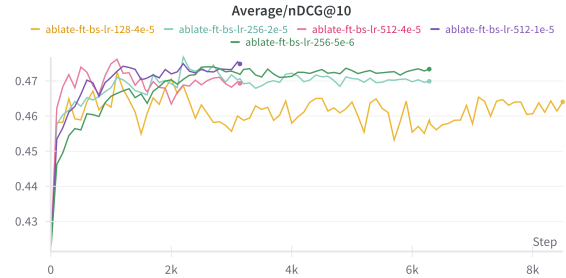


Figure 11: Hyperparameter sweep for different batch sizes and learning rates.

pected insensitivity to presumably important hyperparameters, as was the case when we experimented with various batch size settings and learning rates for fine-tuning as shown in Figure 11. Other times, we were surprised by unexpected sensitivity, like the unforeseen differences in trajectory between shorter and longer training runs during (see the comparisons in Table 8 and Figure 10 for example – where we see that training for more steps with our linear learning rate decay schedule leads to much worse performance at the beginning of training).

Although we ran many informal experiments while developing Arctic embed, we believe there is still much to be learned about the peculiarities of tuning text embedding training.

E Full MTEB Score Breakdown

	baseline	using e5-base- unsupervised	using nomic data	using se- quence length 128	4k batch 80k step stratified	4k batch 80k step unstratified	4k batch 20k step stratified	4k batch 20k step unstratified	without source strat- ification	baseline, finetuned	using e5-base- unsupervised, finetuned	using nomic data, fine- tuned
ArguAna	45.14	43.95	53.66	39.69	44.13	39.70	44.75	42.18	45.65	58.77	55.44	51.78
CQADupstackRetrieval	42.26	42.24	41.57	40.80	40.71	37.36	40.15	36.90	39.05	43.24	44.11	42.75
ClimateFEVER	20.35	23.75	22.09	21.33	20.73	19.51	18.47	19.78	18.79	35.55	38.82	32.57
DBPedia	38.97	37.83	38.35	36.92	38.77	37.58	39.27	35.77	37.02	44.68	45.81	43.07
FEVER	71.65	71.39	69.59	64.42	70.61	71.04	72.55	68.34	68.82	87.55	88.46	86.78
FiQA2018	42.09	45.20	37.40	40.46	39.70	35.30	38.73	34.92	37.67	40.44	42.88	36.79
HotpotQA	57.77	53.97	60.52	56.69	59.10	47.22	60.54	47.22	46.27	71.77	73.03	69.84
MSMARCO	33.85	33.75	34.35	33.77	33.12	31.46	32.79	31.05	32.40	41.73	41.89	40.26
NFCorpus	33.83	36.28	36.25	32.83	34.28	32.71	32.81	31.90	32.58	34.63	36.16	34.98
NQ	46.92	47.07	48.46	46.68	45.81	37.30	44.87	37.55	38.85	60.37	61.82	58.48
QuoraRetrieval	87.20	87.45	88.13	87.30	86.99	86.25	86.51	86.10	86.58	87.32	87.60	87.94
SCIDOCS	20.39	21.54	21.62	20.24	20.02	17.88	19.09	18.30	18.58	20.20	21.09	21.00
SciFact	69.42	70.97	74.97	68.90	68.68	65.99	66.89	66.88	68.36	70.58	73.34	75.16
TRECCOVID	71.32	66.99	50.91	70.96	66.82	62.14	61.23	59.14	63.51	79.98	80.27	74.70
Touche2020	23.38	21.97	20.36	22.00	22.42	20.34	21.76	20.22	22.03	31.95	29.38	27.40
Overall Average	46.97	46.96	46.55	45.53	46.13	42.79	45.36	42.42	43.74	53.92	54.67	52.23

Table 9: Full Per Dataset NDCG@10 MTEB Retrieval scores by dataset for each ablation variant.